Занятие 45.2. Динамические массивы

Помимо **динамического выделения переменных** мы также можем динамически выделять и массивы. В отличие от **фиксированного массива**, где его размер должен быть известен во время компиляции, динамическое выделение массива в языке C++ позволяет нам устанавливать его длину во время выполнения программы.

Оглавление:

1. Динамические массивы
2. Удаление динамического массива
3. Инициализация динамических массивов
4. Изменение длины массивов
5. Тест

Динамические массивы

Для выделения динамического массива и работы с ним используются отдельные формы операторов new и delete: `new[]` и `delete[]`.

```
#include <iostream>

int main()
{
    std::cout << "Enter a positive integer: ";
    int length;
    std::cin >> length;

    int *array = new int[length]; // используем оператор new[] для выделения массива. Обратите внимание, переменная length м

    std::cout << "I just allocated an array of integers of length " << length << '\n';

    array[0] = 7; // присваиваем элементу под индексом 0 значение 7

    delete[] array; // используем оператор delete[] для освобождения выделенной массиву памяти
    array = 0; // используйте nullptr вместо 0 в C++11

    return 0;
}
```

Поскольку мы выделяем массив, то C++ понимает, что он должен использовать другую форму оператора new — форму для массива, а не для переменной. По факту вызывается оператор `new[]`, даже если мы и не указываем `[]` сразу после ключевого слова `new`.

Обратите внимание, поскольку память для динамических и фиксированных массивов выделяется из разных «резервуаров», то размер динамического массива может быть довольно большим. Вы можете запустить программу, приведенную выше, но уже выделить массив длиной 1 000 000 (или, возможно, даже 100 000 000) без проблем. Попробуйте!

Удаление динамического массива

При удалении динамических массивов также используется форма оператора delete для массивов — `delete[]`. Таким образом, мы сообщаем процессору, что ему нужно очистить память от нескольких переменных вместо одной. Самая распространенная ошибка, которую совершают новички при работе с динамическим выделением памяти, является использование delete вместо `delete[]` для удаления динамических массивов. Использование формы оператора delete для переменных при удалении массива приведет к таким неожиданным результатам, как повреждение данных, утечка памяти, сбой или другие проблемы.

Инициализация динамических массивов

Если вы хотите инициализировать динамический массив значением `0`, то всё довольно просто:

```
int *array = new int[length]();
```

До C++11 не было простого способа инициализировать динамический массив ненулевыми значениями (список инициализаторов работал только с фиксированными массивами). А это означает, что нужно перебрать каждый элемент массива и присвоить ему значение явным указанием:

```
int *array = new int[5];
array[0] = 9;
array[1] = 7;
array[2] = 5;
array[3] = 3;
array[4] = 1;
```

Немного утомляет, не правда ли?

Однако, начиная с C++11, появилась возможность инициализации динамических массивов через списки инициализаторов:

```
int fixedArray[5] = { 9, 7, 5, 3, 1 }; // инициализируем фиксированный массив
int *array = new int[5] { 9, 7, 5, 3, 1 }; // инициализируем динамический массив
```

Обратите внимание, в синтаксисе динамического массива между длиной массива и списком инициализаторов оператора присваивания (`=`) нет.

В C++11 фиксированные массивы также могут быть инициализированы с использованием uniform-инициализации:

```
int fixedArray[5] { 9, 7, 5, 3, 1 }; // инициализируем фиксированный массив в C++11
char fixedArray[6] { "Hello, world!" }; // инициализируем фиксированный массив в C++11
```

Однако, будьте осторожны, так как в C++11 вы не можете инициализировать динамический массив символов строкой C-style:

```
char *array = new char[6] { "Hello, world!" }; // не работает в C++11
```

Вместо этого вы можете динамически выделить `std::string` (или выделить динамический массив символов, а затем с помощью функции strcpy_s() скопировать содержимое нужной строки в этот массив).

Также обратите внимание на то, что динамические массивы должны быть объявлены с явным указанием их длины:

```
int fixedArray[] {1, 2, 3}; // ок: явное указание длины фиксированного массива

int *dynamicArray1 = new int[] {1, 2, 3}; // не ок: неявное указание длины динамического массива

int *dynamicArray2 = new int[3] {1, 2, 3}; // ок: явное указание длины динамического массива
```

Изменение длины массивов

Динамическое выделение массивов позволяет задавать их длину во время выделения. Однако C++ не предоставляет встроенный способ изменения длины массива, который уже был выделен. Но и это ограничение можно обойти, динамически выделив новый массив, скопировав все элементы из старого массива, а затем удалив старый массив. Однако этот способ подвержен ошибкам (об этом чуть позже).

К счастью, в языке C++ есть массивы, размер которых можно изменять, и называются они векторами (std::vector). О них мы поговорим на соответствующем уроке.

Тест

Напишите программу, которая:

- спрашивает у пользователя, сколько имен он хочет ввести;
- просит пользователя ввести каждое имя;
- вызывает функцию для сортировки имен в алфавитном порядке;
- выводит отсортированный список имен.

> ***Подсказки:***
>
> - Используйте динамическое выделение std::string для хранения имен.
> - std::string поддерживает сравнение строк с помощью операторов сравнения `<` и `>`.

Пример результата выполнения вашей программы:

```
How many names would you like to enter? 5
Enter name #1: Jason
Enter name #2: Mark
Enter name #3: Alex
Enter name #4: Chris
Enter name #5: John

Here is your sorted list:
Name #1: Alex
Name #2: Chris
Name #3: Jason
Name #4: John
Name #5: Mark
```

Функция для сортировки имен в алфавитном порядке

```
#include <iostream>
#include <algorithm> // для std::swap. В C++11 используйте заголовок <utility>

int main()
{
    const int length = 5;
    int array[length] = { 30, 50, 20, 10, 40 };

    // Перебираем каждый элемент массива (кроме последнего, он уже будет отсортирован к тому времени, когда мы до него добера
    for (int startIndex = 0; startIndex < length - 1; ++startIndex)
    {
        // В переменной smallestIndex хранится индекс наименьшего значения, которое мы нашли в этой итерации.
        // Начнем с того, что первый элемент в этой итерации - это первый элемент (индекс 0)
        int smallestIndex = startIndex;

        // Цикл для поиска элемента в оставшейся части массива
        for (int currentIndex = startIndex + 1; currentIndex < length; ++currentIndex)
        {
            // Если мы нашли элемент, который меньше нашего наименьшего элемента,
            if (array[currentIndex] < array[smallestIndex])
                // то запоминаем его
                smallestIndex = currentIndex;
        }

        // smallestIndex теперь наименьший элемент.
        // Поменяем местами наше начальное наименьшее число с тем, которое мы обнаружили
        std::swap(array[startIndex], array[smallestIndex]);
    }

    // Теперь, когда всё сделали, распечатаем
    for (int index = 0; index < length; ++index)
        std::cout << array[index] << " ";

    return 0;
}
```

Ответ

```
#include <iostream>
#include <string>
#include <utility> // для std::swap(). Если у вас не поддерживается C++11, то тогда #include <algorithm>

void sortArray(std::string *array, int length)
{
    // Перебираем каждый элемент массива
    for (int startIndex = 0; startIndex < length; ++startIndex)
    {
        // smallestIndex - индекс наименьшего элемента, с которым мы столкнулись
        int smallestIndex = startIndex;

        // Ищем наименьший элемент, который остался в массиве (начиная со startIndex+1)
        for (int currentIndex = startIndex + 1; currentIndex < length; ++currentIndex)
        {
            // Если текущий элемент меньше нашего ранее найденного наименьшего элемента,
            if (array[currentIndex] < array[smallestIndex])
                // то запоминаем его новым наименьшим значением в этой итерации
                smallestIndex = currentIndex;
        }

        // Поменяем местами наш начальный элемент с найденным наименьшим элементом массива
        std::swap(array[startIndex], array[smallestIndex]);
    }
}

int main()
{
    std::cout << "How many names would you like to enter? ";
    int length;
    std::cin >> length;

    // Выделяем массив для хранения имен
    std::string *names = new std::string[length];

    // Просим пользователя ввести все имена
    for (int i = 0; i < length; ++i)
    {
        std::cout << "Enter name #" << i + 1 << ": ";
        std::cin >> names[i];
    }

    // Сортируем массив
    sortArray(names, length);

    std::cout << "\nHere is your sorted list:\n";
    // Выводим отсортированный массив
    for (int i = 0; i < length; ++i)
        std::cout << "Name #" << i + 1 << ": " << names[i] << '\n';

    delete[] names; // не забываем использовать оператор delete[] для освобождения памяти
        names = nullptr; // используйте 0, если не поддерживается C++11

    return 0;
}
```